第 5-3 讲 • 隐私与离线：这个项目的结构性优势

> **模块 5 • 质量与安全** ｜ 建议用时 **40 分钟**（阅读 24 + 动手 16）
> 前置要求：完成 5-1、5-2

本节目标

读完并做完这一讲，你能：

1. 分清**"承诺不收集"**和**"结构上收不到"**——这是本讲最重要的区分。
2. 用一条命令**自己证明**这个项目没有联网代码（而不是听我说）。
3. 说出全项目仅有的 **3 处网络相关调用**，以及各自为什么是安全的。
4. 理解**本地 OCR** 意味着什么（截图识别为什么不上传）。
5. 能向别人**讲清楚**这个软件的隐私优势（这是可以拿去宣传的真本事）。

一、最重要的一件事：承诺 vs 结构

1.1 两种"隐私安全"

市面上的软件说"我们保护你的隐私"，通常有两种情况：

	承诺型	**结构型**
说法	"我们**承诺**不上传你的数据"	"我们**收不到**你的数据"
依赖	**对方的人品和制度**	**技术架构**
怎么验证	只能信	**可以自己查**
变了怎么办	改个隐私政策就能变	要改架构，**藏不住**

记账簿属于第二种。

为什么这很重要：承诺型的软件，**明天换个老板、改了政策，你的数据就可能被上传**——而你只能事后从新闻里知道。**结构型的软件，你可以现在就自己验证。**

1.2 一个残酷的现实

很多"记账 App"是这样的：

```
你的手机 → 上传到服务器 → 服务器存一份 → 后台可以看到全部账目
                                ↓
                          还可能卖给第三方做用户画像
```

这不是阴谋论，是商业模式：免费 App 靠什么活？**靠数据或者广告。**

记账簿的商业模式很简单：**没有商业模式。**它不联网，所以**没法靠数据赚钱**，也就**没有动机去收数据。**

"没有动机"比"承诺不做"可靠得多。

二、自己动手证明（本讲核心）

不要相信我说的话。 下面这条命令会扫出全项目所有网络调用：

```
cd "C:\Users\34123\Desktop\记账簿"
Get-ChildItem src -Recurse -Include *.ts,*.tsx |
  Select-String "fetch\(|axios|XMLHttpRequest|https\.get|http\.get"
```

我实测的结果——全项目只有 3 处，我逐个给你解释。

2.1 第一处：检查更新（`security.service.ts:86`）

```
const response = await fetch(url, { redirect: 'follow' })
```

这是"检查更新"功能（设置 → 关于 → 检查更新）。

为什么它不算"偷偷联网"：

事实	说明
只有用户主动点击才触发	不会在后台自动跑
请求的是静态版本文件	只是一个 JSON，形如 `{"version":"1.1.0"}`
不携带任何用户数据	没有账号、没有设备标识、没有账目内容
失败也不影响使用	我实测过：连不上时返回"检查失败"，软件照常用

关键判断：它发的是"我要看看有没有新版本"，不是"这是我的数据"。

2.2 第二处：`ledger://` 协议（`protocol.ts:37`）

```
// 越界保护：只允许读数据目录内的文件
if (!target.startsWith(dataDir + path.sep)) {
  return new Response('越界访问被拒绝', { status: 403 })
}
if (!existsSync(target)) {
  return new Response('文件不存在', { status: 404 })
}
return net.fetch(pathToFileURL(target).toString())
```

这个 `net.fetch` 看起来像网络请求，实际上读的是本地文件。

它在做什么：界面上要显示收据图片，但**界面进程碰不到文件系统**（2-1 讲）。所以主进程注册了一个自定义协议 `ledger://`，界面用 `ledger://xxx.jpg` 这样的地址请求图片，主进程读出来给它。

两个安全细节值得学：

细节	防什么
`target.startsWith(dataDir)` **检查**	**越界访问**：防止界面请求 `ledger://../../Windows/System32/config` 这种路径，去读数据目录之外的文件
文件不存在返回 404	正常处理，不崩溃

这是"路径穿越漏洞"的标准防护。记住这个模式：任何"按用户提供的路径读文件"的功能，都必须检查路径是否在允许的范围内。

2.3 第三处：测试脚本（`selftest.ts:163`）

```
const response = await net.fetch(attachmentUrl(attachment.filePath))
check('ledger:// 协议能读到附件', ...)
```

这是测试代码——它在验证上面那个 `ledger://` 协议工作正常。**同样是读本地文件，而且只在跑测试时执行。**

2.4 结论：渲染层零网络调用

再单独验证一遍界面部分：

```
Get-ChildItem src\renderer -Recurse -Include *.ts,*.tsx | Select-String "fetch\(|axios|XML
```

我实测的结果：没有任何输出。

这意味着：**你看到的每一个界面，都不具备"发网络请求"的能力。** 它们只能通过 `window.api` 请主进程做事（2-2 讲的白名单）。

而你已经在 2-2 讲验证过：那个白名单里 **60 个方法，没有一个能执行系统命令或直接写文件。**

三、三道技术防线

除了"没有联网代码"，项目还有三道技术约束。

3.1 第一道：禁止界面跳转到外部网站

真实代码（`window.ts:42-44`）：

```
win.webContents.setWindowOpenHandler(({ url }) => {
  if (url.startsWith('http://') || url.startsWith('https://')) void shell.openExternal(url
  return { action: 'deny' }        // ★ 一律拒绝在应用内打开
})
```

它在做什么：

情况	处理
界面里点到 http/https 链接	**用系统浏览器打开**（不在应用里）
其它任何新窗口请求	**一律拒绝**

为什么重要：如果应用内能加载网页，那么**一个恶意网页就能拿到界面里的上下文。**现在**应用内根本没有"浏览器"这个出口。**

3.2 第二道：`contextIsolation` + `nodeIntegration: false`

这两条 2-1 讲讲过：

配置	作用
`nodeIntegration: false`	界面**不能**用 `require('fs')` 读文件
`contextIsolation: true`	界面**看不见** preload 内部的变量
白名单（2-2 讲）	界面**只能用**那 60 个方法

三道叠加：界面被关在玻璃房里，**只能通过指定的窗口跟外面说话。**

3.3 第三道：本地 OCR（截图识别不上传）

这是隐私方面最值得说的一条。

先看证据——OCR 模型就在你的项目里（本地文件）：

文件	大小
`ch_PP-OCRv4_det_infer.onnx`（文字检测）	4.53 MB
`ch_PP-OCRv4_rec_infer.onnx`（文字识别）	10.35 MB
`ch_ppocr_mobile_v2.0_cls_infer.onnx`（方向分类）	0.56 MB
`ppocr_keys_v1.txt`（字典）	0.03 MB

加起来 15.5 MB，全部随安装包发到用户机器上。

代码怎么用的（`billshot.ocr.ts`）：

```
import * as ort from 'onnxruntime-node'
...
let detSession: ort.InferenceSession | null = null
let recSession: ort.InferenceSession | null = null
```

`onnxruntime-node` 是一个**本地推理引擎**——它在**你的电脑上**跑模型，**不需要联网。**

对比一下"云端识别"：

	云端 OCR	**记账簿（本地 OCR）**
图片去哪	**上传到服务器**	**不出你的电脑**
识别在哪算	别人的服务器	**你的 CPU**
需要联网	**必须**	**不需要**
别人能看到你的账单吗	**能**	**不能**

这就是那 15.5 MB 的代价换来的东西。 也是为什么安装包有 116 MB——**模型和运行环境都打包进去了，所以断网也能识别。**

这笔交易划不划算：

代价	**换来的**
安装包大 15 MB+	**截图永远不离开你的电脑**
识别速度取决于你的 CPU	**不需要网络，不依赖别人的服务器**
	不需要为识别付费（云端 OCR 通常按次收费）

对记账软件来说，这笔交易非常划算——账单截图是最敏感的数据之一。

四、怎么向别人讲清楚

这是本讲的实用产出：你以后要向朋友、用户、甚至投资人解释这个软件，需要一段能说清的话。

4.1 一句话版本

> **"它不联网，所以收不到你的数据——这不是承诺，是结构决定的。"**

4.2 三句话版本

> ① **没有账号、没有服务器**：记账簿全程在你电脑上跑，数据就是你自己目录里的一个文件。
> ② **截图识别也在本地**：账单截图不出你的电脑，识别用的是随软件一起装的离线模型。
> ③ **你可以自己验证**：全项目的网络调用只有一处（你手动点"检查更新"时），代码是开源的，自己搜得到。

4.3 五句话版本（加上对比）

> 大多数记账 App 要联网，因为它们的账存在自己服务器上——这本身没问题，但意味着**公司能看到你的全部账目**。
> 记账簿的做法相反：**账存你自己的电脑，识别也在你自己的电脑上算**。
> 代价是**不能多设备同步**（同步必须有服务器）。
> 换来的是**没有任何人能拿到你的数据**，包括我们。
> 而且这不是口头承诺——**软件里根本没有联网的代码路径**，你可以自己查。

4.4 一个诚实的补充（必须说）

要主动说明两条边界，否则就是误导：

边界	说明
应用锁 ≠ 整库加密	PIN 挡的是"随手打开"，把 `.db` 拷走仍能读（5-2 讲详细说过）
本地 ≠ 不会丢	数据只在这一台电脑上，硬盘坏了就没了。**必须自己备份到这台电脑之外**

主动说出边界，反而更可信。 含糊其辞的"我们很安全"，会让懂行的人立刻不信任你。

五、动手任务

步骤 1：自己扫一遍网络调用（最重要的一步）

```
cd "C:\Users\34123\Desktop\记账簿"
Get-ChildItem src -Recurse -Include *.ts,*.tsx |
  Select-String "fetch\(|axios|XMLHttpRequest|https\.get|http\.get" |
  ForEach-Object { "$($_.Filename):$($_.LineNumber)" }
```

预期看到 3 处（检查更新、协议读文件、测试）。**对照第 2 节逐个确认它们是什么。**

再单独扫界面部分：

```
Get-ChildItem src\renderer -Recurse -Include *.ts,*.tsx | Select-String "fetch\(|axios|XML
```

预期：没有输出。

这两条命令就是你自己的"隐私审计"。 你现在**不需要相信任何人的承诺**了。

步骤 2：确认 OCR 是本地跑的

```
# 模型文件真的在本地
Get-ChildItem resources\ocr\models | Select-Object Name, @{n='MB';e={[math]::Round($_.Leng


# 用的是本地推理引擎
Select-String -Path src\main\services\billshot.ocr.ts -Pattern "onnxruntime|InferenceSessi
```

预期：看到 4 个模型文件 + `onnxruntime-node` 的导入。

再做一个实验——断网识别：

1. **断开网络**（关掉 WiFi / 拔网线）。
2. 打开记账簿 → 截图记账。
3. **选一张账单截图，开始分析。**

预期：**识别正常工作。** 因为整个流程都在本地。

这个实验的感受是最直接的——你亲眼看到"没网也能识别"。

步骤 3：验证"禁止跳转外链"

```
Select-String -Path src\main\window.ts -Pattern "setWindowOpenHandler" -Context 0,4
```

读这段代码，回答：如果在界面里点了 http 链接，会发生什么？

答案：`shell.openExternal(url)` —— **用系统默认浏览器打开**，**不是在应用里加载。**而其它任何新窗口请求都被 `deny`。

步骤 4：验证"越界保护"（`ledger://` 的防护）

```
Select-String -Path src\main\protocol.ts -Pattern "startsWith|越界|403" -Context 1,2
```

读懂它之后回答：

- 这个检查在防什么？
- 如果没这个检查，会有什么后果？

答案：

- 防**路径穿越**：界面如果请求 `ledger://../../Windows/System32/xxx`，就能读到数据目录之外的文件
- 没这个检查的话，**界面（以及任何能影响界面的东西）就有了一条读取任意文件的通道**

这是一个真实的漏洞类型，很多 Electron 应用都犯过这个错。

步骤 5：写下你自己的"三句话介绍"

任务：不看讲义，用你自己的话写三句话，向一个不懂技术的朋友解释这个软件的隐私优势。

写完对照 4.2 节，看有没有漏掉关键点：

- 有没有说"数据在你自己电脑上"？
- 有没有说"截图识别也在本地"？
- 有没有说"可以自己验证"？

这个练习的价值：**能讲清楚，才算真懂。** 而且这段话你以后真的会用上（给朋友推荐、写产品介绍）。

卡住了怎么办

现象	原因	怎么办
搜索出更多网络调用	可能匹配到了注释或字符串	打开那个文件看看，判断是不是真调用
断网后截图识别失败	可能是模型文件缺失	检查 `resources\ocr\models` 有没有文件
不理解"越界保护"	需要一点文件路径知识	回顾 1-2 讲的路径概念
想验证"真的不上传"	好想法	可以用抓包工具（如 Fiddler）监控，会发现只有你点"检查更新"时才有请求

六、自测题

1. "承诺不收集"和"结构上收不到"有什么区别？为什么后者更可信？

2. 全项目的 3 处网络相关调用分别是什么？为什么都不算"偷偷联网"？

3. `ledger://` 协议的那个 `startsWith(dataDir)` 检查在防什么？

4. 本地 OCR 的代价和收益分别是什么？

5. 向别人介绍这个软件时，必须主动说明哪两条边界？为什么？

答案与解析

1. 承诺型依赖"对方的人品和制度"，**你只能相信**，而且对方改个政策就能变。**结构型**依赖"技术架构"，**你可以自己验证**（搜代码、断网测试），而且**要变就得改架构，藏不住。** → 记账簿属于结构型：**没有联网的代码路径。**

2. ① **检查更新**（`security.service.ts`）：只有用户主动点击才触发，请求的是静态版本文件，不携带任何用户数据；② **`ledger://` 协议**（`protocol.ts`）：读的是**本地文件**（用于显示收据图片），而且有越界保护；③ **测试脚本**（`selftest.ts`）：只在跑测试时执行的本地读文件。 → 三处都不涉及"把用户数据发出去"。

3. 路径穿越攻击：防止界面请求 `ledger://../../Windows/System32/config` 这类路径，去读**数据目录之外**的文件。 → 通用原则：**任何"按用户提供的路径读文件"的功能，都必须检查路径是否在允许范围内。**

4. 代价：安装包大 15 MB+、识别速度取决于本机 CPU、需要自己带模型文件。**收益**：截图**永远不离开你的电脑**、不需要联网、不需要按次付费。 → 对记账软件来说**非常划算**——账单截图是最敏感的数据之一。

5. ① **应用锁不是整库加密**（PIN 只防"随手打开"，拷走 `.db` 仍能读）；② **数据只在这一台电脑上**（硬盘坏了就没了，必须自己备份到电脑之外）。 → **因为主动说出边界反而更可信。**含糊其辞的"我们很安全"会让懂行的人立刻不信任你。

七、术语表

术语	一句话解释
结构型隐私	靠技术架构保证收不到数据，而非靠承诺
承诺型隐私	靠公司政策承诺不收集，依赖对方守信
本地推理 / Local Inference	在本机跑 AI 模型，不需要联网
ONNX	一种通用的模型文件格式
本地 OCR	在本机做文字识别，图片不外发
自定义协议	自己注册的地址格式（如 `ledger://`），用于进程间安全取文件
路径穿越 / Path Traversal	用 `../` 跳出允许目录去读别的文件
越界保护	检查请求的路径是否在允许范围内
纵深防御	多层安全措施叠加，不依赖单一防线
隐私政策	公司对数据处理的书面承诺（**可以被修改**）

八、本讲与项目的对应

- 你查看的文件：`security.service.ts`、`protocol.ts`、`billshot.ocr.ts`、`window.ts`

- 你做的实验：扫全项目网络调用、断网跑截图识别
- **本讲不修改任何代码，不影响你的记账数据**
- 上一讲：5-2 数据安全 ｜ **下一讲进入模块 6：6-1 从源码到安装包**

九、模块 5 小结

讲	你学到的
5-1	**验证的价值**：15 个命令、88 项检查在防"改 A 坏 B"
5-2	**数据安全四层**：软删除、附件保留、自动备份、约束与迁移保护
5-3	**结构型隐私**：没有联网代码、本地 OCR、自己可验证

模块 5 结束时，你已经能回答两个专业问题：

1. **"你怎么保证改动没有破坏别的东西？"** → 自动化验证 + 按范围选择检查
2. **"你的软件凭什么说隐私安全？"** → 结构上收不到，而且我能给你看证据

这两个问题的答案，就是"能独立开发"和"能写代码"的分界线。 前者知道怎么保证质量，后者只是能让程序跑起来。

讲义版本：v1 ｜ 对应项目版本：0.1.0